

AI FITNESS TRAINER

A Computer Vision Based Real-Time Exercise Monitoring System

Project Details	
Academic Year	2026
Student Name	ARGHAJIT SAHA
Roll / Enrollment No.	24BAI10234
Department	CSE AIML
Institution	VELLORE INSTITUTE OF TECHNOLOGY, BHOPAL

This report presents the design, implementation, evaluation methodology, testing, and application output of the AI Fitness Trainer.

1. Introduction & Problem Definition

Fitness exercises such as curls, squats, and push-ups require controlled range of motion and consistent movement patterns. In unsupervised workouts, manual repetition counting and visual form checking are difficult to maintain continuously. The project focuses on an accessible webcam-based software system that extracts human pose information, converts it into interpretable joint angles, counts repetitions, and retains workout history.

1.1 Problem Statement

There is a need for a lightweight fitness-monitoring system that can operate with ordinary camera hardware and provide immediate movement feedback without dedicated wearable sensors. The system processes video, detects body landmarks, evaluates exercise-specific kinematics, distinguishes completed movement cycles from incomplete ones, and preserves session information.

1.2 Objectives

- Detect 33 body landmarks using MediaPipe Pose Landmarker.
- Calculate joint angles from landmark coordinates using vector geometry.
- Track Bicep Curl, Squat, and Push-Up repetitions with exercise-specific FSM logic.
- Provide real-time telemetry and visual feedback through a web interface.
- Authenticate users and maintain user-specific workout history in SQLite.
- Verify core mathematics and application workflows through automated and manual testing.

1.3 Scope

The current scope covers local webcam input, pose estimation, kinematic analysis, repetition tracking, live telemetry, authenticated session storage, workout history, and browser-based dashboards. It is a fitness tracking and movement-verification tool, not a medical diagnostic or clinical rehabilitation system.

2. Existing Approaches & Proposed System

Human trainers provide direct observation but require continuous availability. Manual logging records completed workouts but does not verify movement. Wearable sensors automate some measurements but require dedicated hardware. The proposed system uses a standard webcam and computer vision to connect movement analysis with automated workout logging.

Aspect	Traditional / Manual	Proposed AI Trainer
Input	Human observation / manual entry	Webcam video
Analysis	Trainer judgement / user entry	Pose landmarks + joint angles
Rep tracking	Manual	FSM-based automatic counting
Storage	Notebook/app entry	SQLite session records
Interface	Trainer or logging app	Real-time web dashboard

3. Requirements & System Design

3.1 Functional Requirements

- User registration, login, logout, and authenticated access.
- Exercise selection for Bicep Curl, Squat, and Push-Up.
- Webcam capture and continuous frame transmission.
- Pose landmark extraction and joint-angle calculation.
- Exercise-specific repetition counting and feedback.
- Workout completion and session persistence.
- Historical workout and profile analytics.
- Interactive REST API documentation through FastAPI Swagger UI.

3.2 Non-Functional Requirements

Requirement	Implementation / Target
Performance	Real-time processing target around 30 FPS.
Usability	High-contrast live HUD and responsive web UI.
Reliability	FSM hysteresis/debounce reduces threshold noise.
Security	Password hashing, authentication, protected routes.
Maintainability	Modular Python backend and component-based React frontend.
Error Handling	Graceful handling of missing pose and disconnected streams.

3.3 High-Level Architecture

The browser captures camera frames, sends them through a persistent WebSocket, and receives telemetry. The backend decodes frames, runs pose estimation, performs kinematic calculations, routes the selected exercise strategy, and returns structured results. Completed sessions are stored in SQLite.

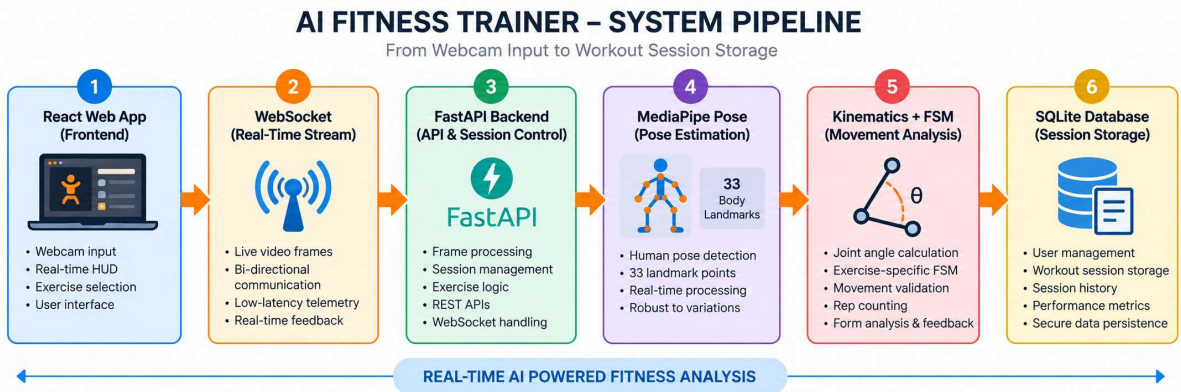


Figure 1. High-level end-to-end architecture

3.4 Database Design

Entity	Key Fields	Purpose
USERS	user_id, username, created_at	Authenticated user identity.
WORKOUT_SESSIONS	session_id, user_id, exercise_type, total_reps, invalid_reps, form_accuracy_pct, duration_seconds, timestamp	Completed workout telemetry linked to a user.

4. Technical Implementation

4.1 Frontend & Backend

The frontend is a React/Vite single-page application with Tailwind CSS and Lucide icons. The backend uses FastAPI for REST routes and a WebSocket endpoint for real-time workout processing.

4.2 Real-Time Webcam Pipeline

- Browser requests webcam permission.
- Frames are captured to a canvas and encoded as JPEG data.
- Frames are transmitted over the active WebSocket.
- FastAPI decodes the frame with OpenCV and passes it to the pose detector.
- Telemetry containing angle, state, repetitions, feedback, and processing information is returned.

4.3 Pose Detection & Kinematic Analysis

MediaPipe Pose Landmarker provides normalized body landmarks. For a joint B, points A-B-C define two vectors. The angle at B is computed from their dot product:

$$\theta = \arccos((BA \cdot BC) / (|BA| |BC|)) \times 180/\pi$$

The resulting angle lies between 0° and 180° and is used by each exercise strategy to identify movement phases.

4.4 Exercise Logic

Exercise	Landmarks	Main thresholds / rule
Bicep Curl	Shoulder → Elbow → Wrist	Extended ≥160°; contracted ≤45°; elbow drift constrained.
Squat	Hip → Knee → Ankle	Standing ≈165°+; depth target ≤90°; ascent completes rep.
Push-Up	Shoulder → Elbow → Wrist	Up ≈155°+; down target ≤105°; visible arm tracked.

4.5 Finite State Machine

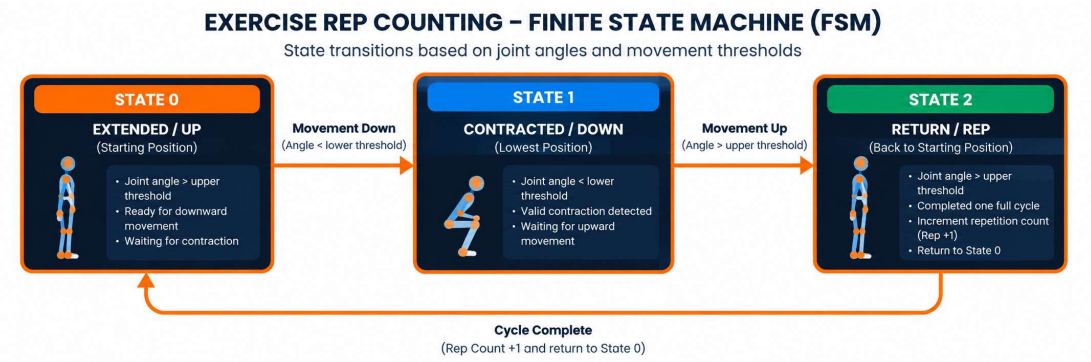


Figure 2. Simplified repetition-counting FSM

5. Dataset, Evaluation Methodology & Limitations

5.1 Dataset

Validation references the public “A Multi-View Raw Video Dataset of Seven Fitness Exercises” hosted on Mendeley Data. The downloaded structure contains exercise folders including bicep_curl, push_up, squat and other exercises, with good/bad quality subfolders and multiple camera perspectives. Evaluation is limited to the three exercises supported by the application.

5.2 Evaluation Protocol

- Select reference videos representing supported exercises and available views.
- Manually count actual repetitions to establish ground truth.
- Process the same videos through the AI Fitness Trainer pipeline.
- Record detected repetitions and compare them with the manual count.
- Report Repetition Detection Rate (RDR) rather than unsupported overall accuracy.

$$\text{RDR} = (\text{Detected Repetitions} / \text{Actual Repetitions}) \times 100\%$$

5.3 Important Evaluation Boundary

The dataset is an offline benchmarking and verification resource. The project does not train a custom convolutional neural network on these videos. Because good/bad labels do not map one-to-one to every exercise-specific rule, controlled recordings can supplement the public dataset for particular shallow-rep or form conditions.

5.4 Known Failure Modes

Condition	Expected behavior / limitation
No visible person	Pose result unavailable; application waits for valid landmarks.
Occlusion	Low visibility can reduce reliable joint tracking.
Partial range of motion	FSM may not complete the rep if the threshold is not reached.
Poor lighting / loose clothing	Landmark confidence and apparent joint positions may degrade.

5.5 Performance Measurement

The system targets real-time operation at approximately 30 FPS. Final performance figures should be taken from measured experimental runs. Live telemetry can be used to record processing behavior.

6. Testing & Verification

6.1 Automated Tests

Test	Purpose	Result
Module/database imports	Verify core dependencies and database module import.	PASSED
Straight angle	Collinear points should produce approximately 180°.	PASSED
Right angle	Perpendicular vectors should produce approximately 90°.	PASSED

Automated execution: 3 tests collected • 3 passed

6.2 Manual Functional Verification

Area	Verification focus
Authentication	Signup, duplicate username/email, invalid email, password mismatch, valid login, logout, protected routes.
Exercise engine	Bicep Curl, Squat, and Push-Up rep lifecycle and feedback.
WebSocket	Browser-to-FastAPI frame stream and telemetry response.
Database	Completed workout linked to authenticated user and available in history.
UI	Dashboard, exercise selection, live workout, history, profile and API documentation.
Negative cases	No person/camera obstruction, abrupt disconnect, incomplete movement.

6.3 Traceability Summary

Testing follows the workflow from input acquisition to stored output: camera → WebSocket → pose landmarks → angle calculation → exercise FSM → telemetry → session persistence → historical display.

6.4 Security & Data Integrity

- Passwords are handled through hashing rather than plaintext storage.
- Protected application routes require an authenticated session.
- Workout records contain a user association.
- API validation and error handling reject malformed requests.

7. Application Output — Authentication & Dashboard

The following are actual application screenshots supplied with the project. They document the final authentication and dashboard interfaces.

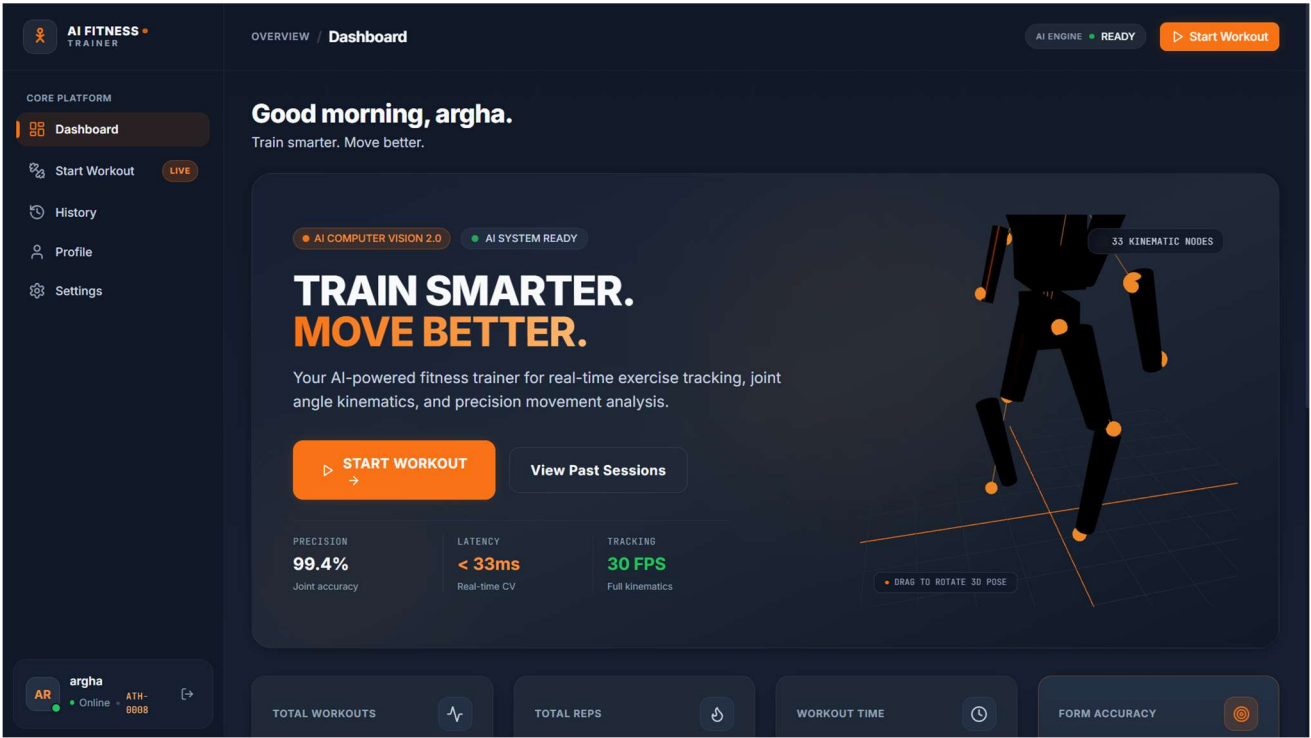


Figure 5. Dashboard overview and workout analytics

8. Application Output — Workout, History & Profile

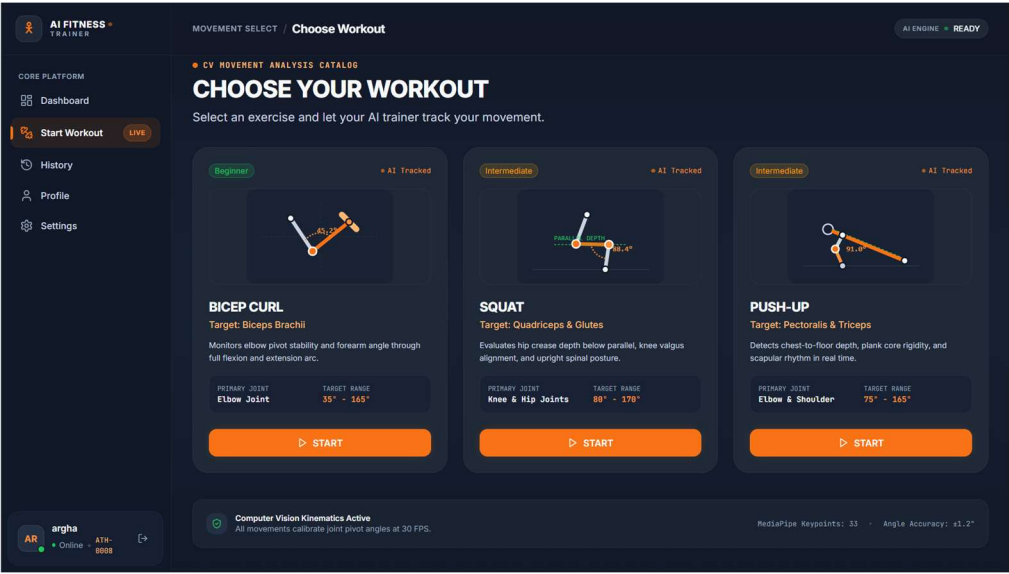


Figure 6. Exercise selection catalog

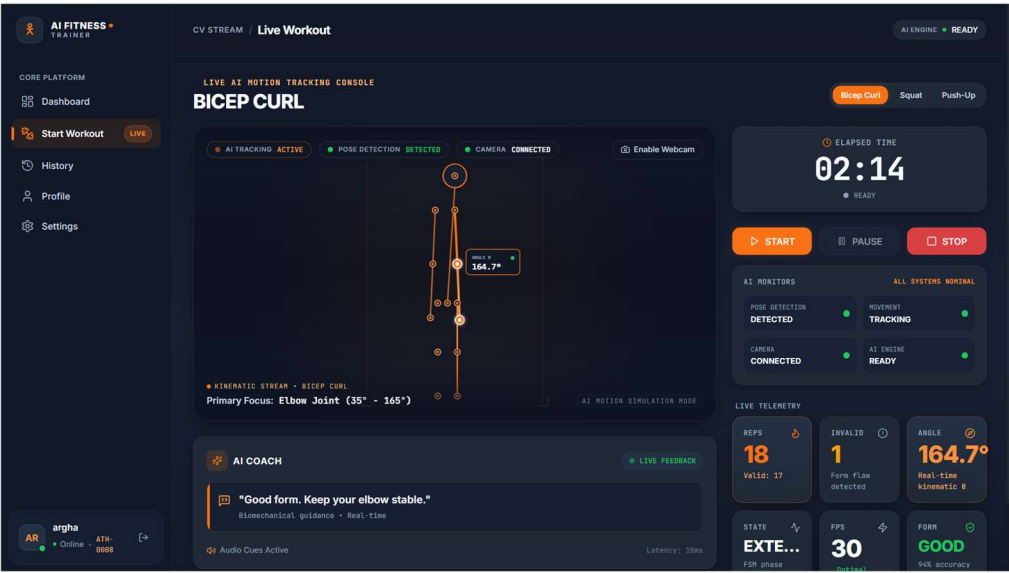
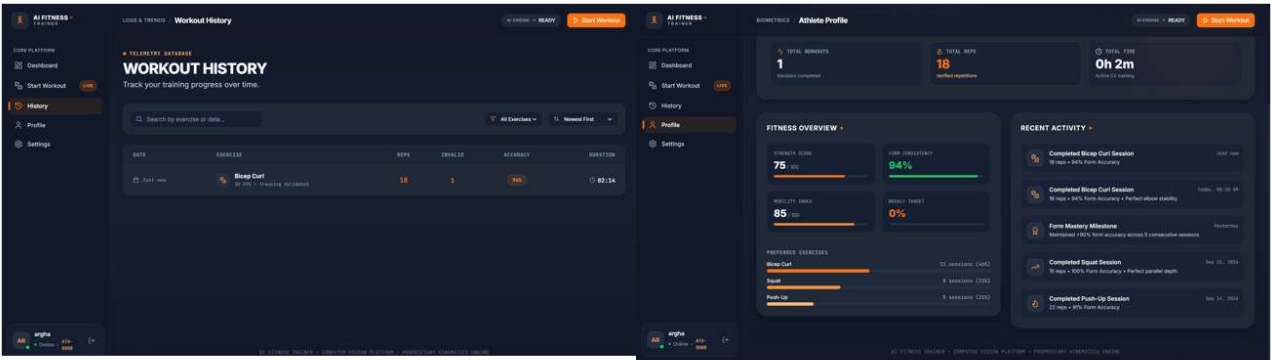


Figure 7. Live AI workout tracking HUD



Figures 8–9. Workout history and athlete profile

9. API Documentation, Challenges & Conclusion

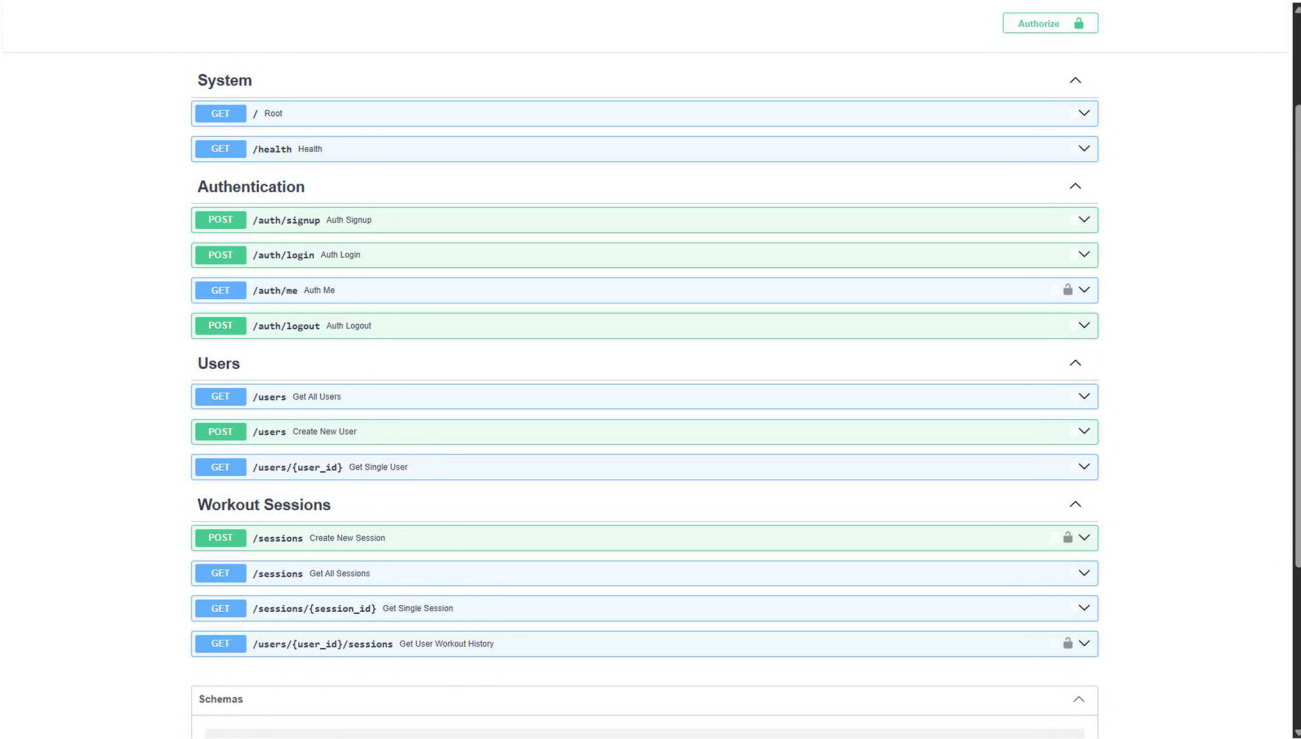


Figure 8. FastAPI Swagger / OpenAPI interface

9.1 Development Challenges & Learnings

- MediaPipe integration required migration to the Tasks-based Pose Landmarker API.
- Real-time processing required coordination between browser capture, WebSocket transport, and backend inference.
- Threshold-based movement detection needed state transitions to reduce accidental double counts.
- Authentication and user-specific history required consistent session identity across frontend and database layers.
- Testing showed the value of isolating mathematical functions before validating the live pipeline.

9.2 Future Scope

Future work can extend the exercise library, add advanced form classification, improve robustness under occlusion and varied camera placement, support multi-person tracking, add voice guidance, and explore mobile/cloud deployment.

9.3 Conclusion

AI Fitness Trainer demonstrates an end-to-end computer-vision workflow connecting webcam input, pose estimation, geometric kinematics, state-based repetition tracking, real-time WebSocket telemetry, authentication, and persistent workout analytics. The current implementation establishes a modular baseline for three exercises and has been verified through automated core tests and application-level testing.

References

MediaPipe Pose Landmarker documentation; OpenCV documentation; FastAPI documentation; React/Vite documentation; SQLite documentation; Pytest documentation; Mendeley Data — “A Multi-View Raw Video Dataset of Seven Fitness Exercises.”